

Security Audit

BoohooBear (Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

BoohooBear collaborated with Hashlock to ensure the security of its Sol smart contract, fostering trust and confidence within its community. Hashlock performed a comprehensive audit, identified vulnerabilities, and recommended necessary changes, which were promptly implemented to safeguard the contract.

Project Context

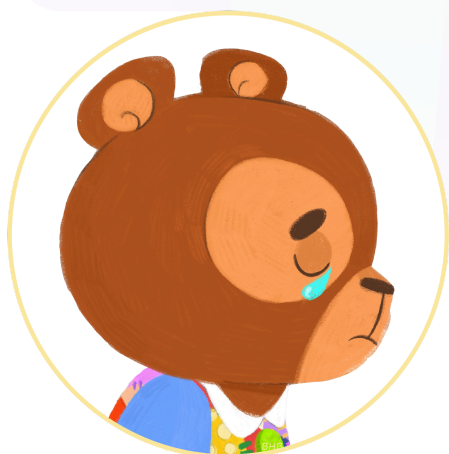
BoohooBear is a meme token on the Binance Smart Chain with ambitions that extend beyond the crypto space. BoohooBear plans to offer physical products for buying and selling, and it aims to develop blockchain technology with an on-chain streaming network for children and a music marketplace platform where artists can create, mint, and sell their music. BoohooBear is committed to honesty, integrity, dignity, and security in all its endeavors. The project not only seeks to educate and encourage children but also to give back through charity work and donations. The BoohooBear development team is dedicated to achieving these goals while creating a safe investment environment free from concerns of rug pulls or misinformation. We're here to build and grow this Bear with you.

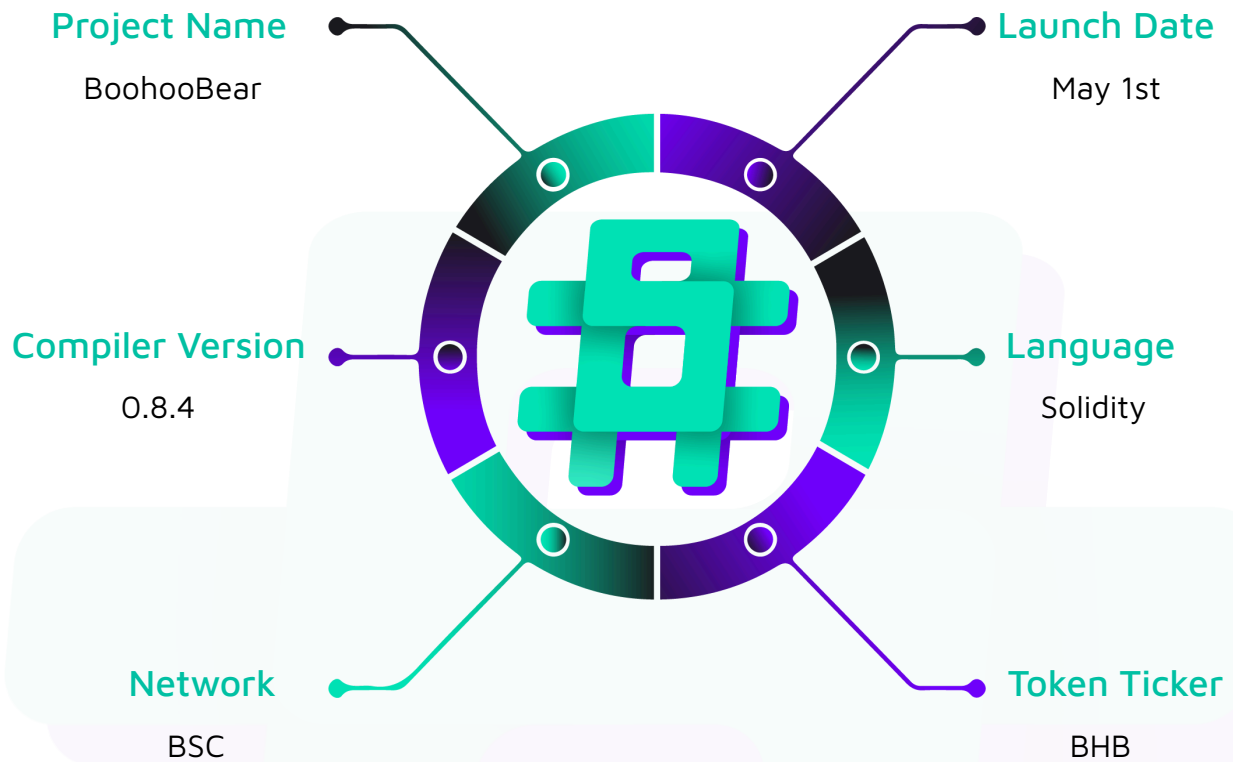
Project Name: BoohooBear

Compiler Version: 0.8.4

Website: <https://BoohooBear.store/>

Logo:



Visualised Context:

Project Visuals:



#hashlock.

Hashlock Pty Ltd

Audit Scope

We at Hashlock audited the solidity code within the BoohooBear project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	BoohooBear Smart Contract
Platform	EVM / Solidity
Audit Date	February, 2025
Contract 1	BoohooBear.sol
Contract 1 MD5 Hash	86cc44ac58b2e171a3a538002b446f4e

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerability

1 Low severity vulnerability

1 Gas Optimisation

3 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BoohooBear.sol ERC20 token contract with fee logic on transfers	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the BoohooBear project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the BoohooBear project smart contract code in the form of a Contract file.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] BoohooBear#_getCurrentSupply - Unbounded loops when iterating over excluded address from the `_excluded` array could cause out of gas error

Description

Because the BoohooBear contract allows for an unlimited number of wallets to be excluded, when the `_getCurrentSupply` function iterates over the excluded addresses during the rate calculation, the contract may revert when a user tries to interact with the contract.

Vulnerability Details

The `_getCurrentSupply` function loops over all excluded addresses to calculate supplies:

```
for (uint256 i = 0; i < _excluded.length; i++) {  
    ...  
}
```

Depending on the number of the excluded addresses that the contract supports, calls to `_getCurrentSupply` may run out of gas, resulting in a denial of service of all external-facing functions.

Impact

The transfer functions may revert due to running out of gas.

Recommendation

It's recommended that the BoohooBear contract allows a limited number of excluded addresses.

Status

Resolved

Low

[L-01] `BoohooBear#excludeFromFee, includeInFee, setTaxFeePercent, setDevelopmentFeePercent, setLiquidityFeePercent, setMaxTxPercent` - Missing events

Description

The above functions are missing events when key states are updated.

Recommendation

Add a relevant event in the function.

Status

Resolved

Gas

[G-01] `BoohooBear` - State variables that could be made constant

Description

The `_developmentWalletAddress`, `_tTotal`, `_name`, `_symbol`, and `_decimals` variables are only set with hardcoded values when deploying the contract.

Recommendation

Make the variables constant.

Status

Resolved

QA

[Q-01] BoohooBear - Floating pragma

Description

The contract has `pragma solidity ^0.8.4` and it might allow the contracts to be deployed with a different version than the one used for testing.

Different pragma versions being used in test and mainnet may pose unidentified security issues.

Recommendation

Specify a specific version of Solidity in the pragma statement.

Status

Resolved

[Q-02] BoohooBear - Redundant use of SafeMath library

Description

In Solidity versions 0.8.0 and above, arithmetic operations such as addition, subtraction, multiplication, and division include built-in overflow and underflow checks by default. This eliminates the need for external libraries like SafeMath, which were previously used to handle these issues in older Solidity versions.

However, the current contract still explicitly uses the SafeMath library (e.g., `using SafeMath for uint256;`). This introduces unnecessary redundancy and inefficiency.

Recommendation

Remove SafeMath usage.

Status

Resolved

[Q-03] BoohooBear#_tokenTransfer - Unnecessary else case

Description

The `_tokenTransfer` function checks the excluded statuses for both the sender and recipient addresses.

However, the last `else` case is never met.

```
if (!_isExcluded[sender] && !_isExcluded[recipient]) {  
    ...  
} else if (!_isExcluded[sender] && _isExcluded[recipient]) {  
    ...  
} else if (!_isExcluded[sender] && !_isExcluded[recipient]) {  
    ...  
} else if (_isExcluded[sender] && _isExcluded[recipient]) {  
    ...  
} else {  
    _transferStandard(sender, recipient, amount);  
}
```

Recommendation

Remove the last `else` case.

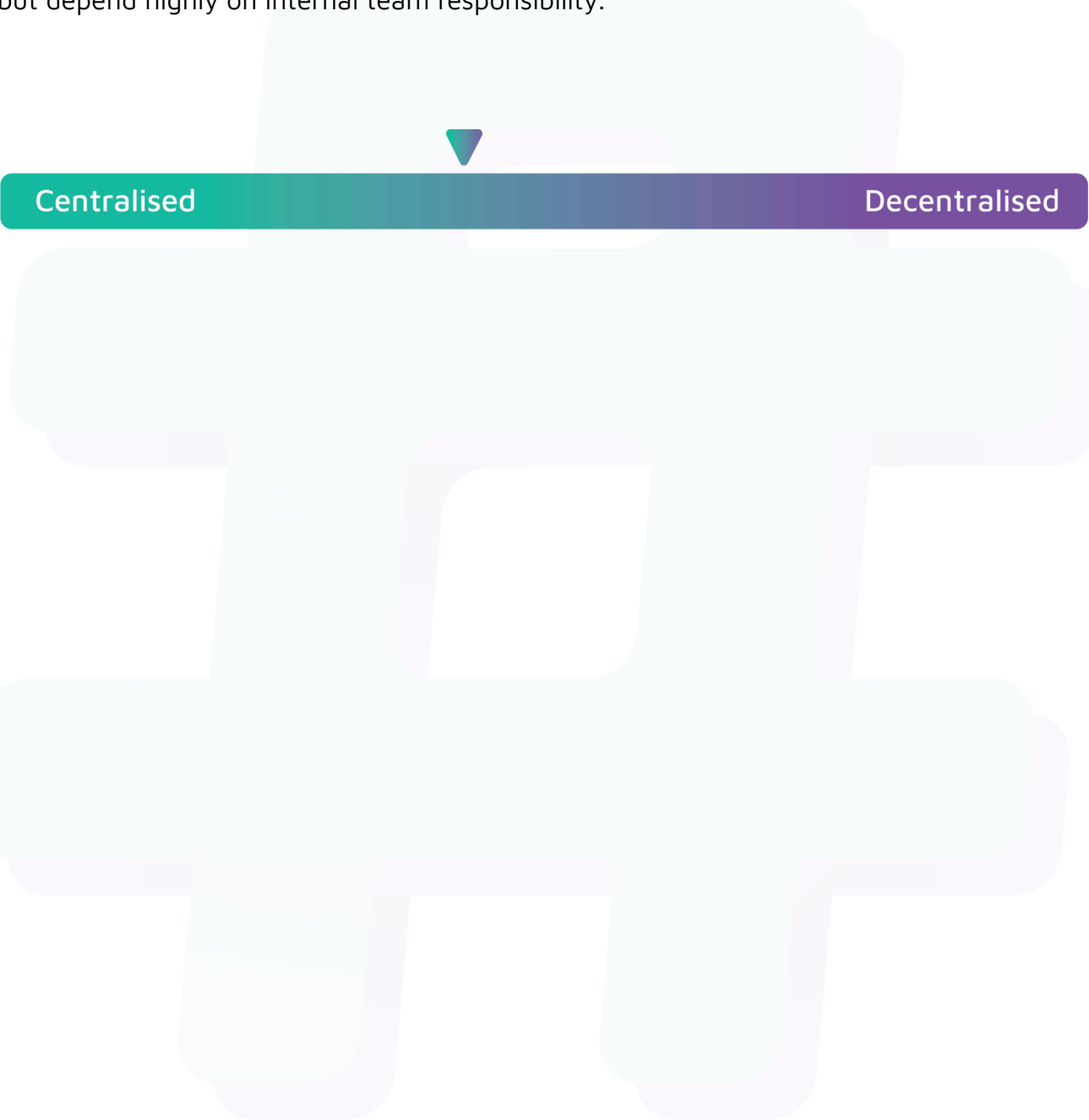
Status

Resolved

Centralisation

The BoohooBear project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the BoohooBear project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.